

Statistical Learning for Hypertension Management

This study focuses on the statistical analysis and predictive modeling of a simulated dataset concerning hypertension management. The data represents adult patients who participated in a 6-month intervention program designed to reduce their systolic blood pressure.

The primary objective of this investigation is to explore, analyze, and model the relationship between the target variable (the reduction in systolic blood pressure, measured in mmHg, after the 6-month period) and a diverse set of covariates. These predictors encompass various patient characteristics, including:

- Clinical factors: age, biological sex, baseline systolic blood pressure, body mass index (BMI), and diabetes status.
- Behavioral factors: smoking status, physical activity, and daily salt intake.
- Treatment-related factors: the assigned treatment arm (Control, Drug A, or Drug B) and the patient's therapy adherence fraction.

Data

```
'data.frame': 700 obs. of 11 variables:
 $ Y          : num  18.9 13.4 44.8 30.6 31 ...
 $ age        : num  62.4 36.8 38.6 58.6 31.9 ...
 $ sex        : Factor w/ 2 levels "Male","Female": 2 1 2 1 1 2 2 2 2 2 ...
 $ baseline_sbp : num  152 140 173 190 162 ...
 $ BMI        : num  29.3 18.7 24.9 22.7 22.1 ...
 $ smoker     : Factor w/ 2 levels "No","Yes": 1 2 1 1 1 1 2 1 1 1 ...
 $ exercise_level: Factor w/ 3 levels "Low","Moderate",...: 1 1 3 1 2 3 2 2 2 2 ...
 $ salt_intake  : num  5.27 8.72 5.86 6.65 6.04 ...
 $ treatment   : Factor w/ 3 levels "Control","DrugA",...: 2 1 3 1 1 3 2 3 2 3 ...
 $ adherence   : num  0.789 0.844 0.535 0.885 0.712 ...
 $ diabetes    : Factor w/ 2 levels "No","Yes": 2 1 2 1 1 2 1 1 1 2 ...
```

Y	age	sex	baseline_sbp
Min. : -97.651	Min. : 30.04	Male : 320	Min. : 99.41
1st Qu.: 5.299	1st Qu.: 42.82	Female: 380	1st Qu.: 138.89
Median : 13.731	Median : 54.28		Median : 148.78
Mean : 14.318	Mean : 54.74		Mean : 149.06
3rd Qu.: 23.403	3rd Qu.: 67.37		3rd Qu.: 159.29
Max. : 60.335	Max. : 79.93		Max. : 199.00

BMI	smoker	exercise_level	salt_intake	treatment
Min. : 15.99	No : 491	Low : 274	Min. : 1.124	Control: 266
1st Qu.: 24.52	Yes: 209	Moderate: 292	1st Qu.: 6.658	DrugA : 203
Median : 26.85		High : 134	Median : 7.960	DrugB : 231
Mean : 26.98			Mean : 7.959	
3rd Qu.: 29.78			3rd Qu.: 9.337	
Max. : 40.74			Max. : 13.433	

adherence	diabetes
Min. : 0.1260	No : 530
1st Qu.: 0.5371	Yes: 170
Median : 0.6798	
Mean : 0.6637	
3rd Qu.: 0.8126	
Max. : 0.9879	

To comprehensively analyze this dataset, we apply a variety of statistical learning techniques, spanning both unsupervised and supervised learning methodologies. For all supervised learning tasks, the dataset is strictly partitioned into a training set (70% of the observations) and a test set (30% of the observations) to properly evaluate model generalization, using a reproducible random seed.

```
n = dim(data.all)[1]
set.seed(1120)
select.train = sample(1:n, n*7/10)
train = data.all[select.train,]
test = data.all[-select.train,]
```

Unsupervised Patient Segmentation (K-Means Clustering)

To understand the inherent structure of the hypertension dataset before applying predictive models, we explore unsupervised patient segmentation using K-means clustering. Since standard K-means relies on Euclidean distance, I subset the data to strictly include continuous numerical covariates, excluding categorical factors.

As an initial baseline, I partitioned the patient data into four distinct groups, or $k=4$, ensuring stable and reproducible results by setting a random seed to 1120 and utilizing 10 random initializations. From this preliminary model, I extracted the core parameters: the cluster centroids (μ_k) representing the average profile of each group, and the specific assignments (r_{ik}) mapping each patient to their respective cluster. To quantify the compactness of this initial grouping, I computed the total within-cluster sum of squares, defined by the loss function $J = \sum_{i=1}^n \sum_{k=1}^K r_{ik} \|x_i - \mu_k\|^2$.

```
set.seed(1120)
x.num = data.all[,c("age","baseline_sbp","BMI","salt_intake","adherence")]
scaled.cov = scale(x.num)
km4 = kmeans(scaled.cov, centers=4, nstart=10)

km4$centers
```

	age	baseline_sbp	BMI	salt_intake	adherence
1	0.26061101	-0.1715193	0.1445699	-0.3331798	-1.3480678
2	0.90858666	0.2343898	-0.1561342	0.6995590	0.3138607
3	-0.93926299	0.4109022	0.3968981	0.3220699	0.1734437
4	-0.07625028	-0.5837516	-0.4410942	-0.8231014	0.6876488

```
km4$cluster[1:20]
```

```
[1] 4 4 3 2 3 1 1 3 3 3 3 4 4 2 4 3 1 1 2 2
```

```
J = km4$tot.withinss
J
```

```
[1] 2362.18
```

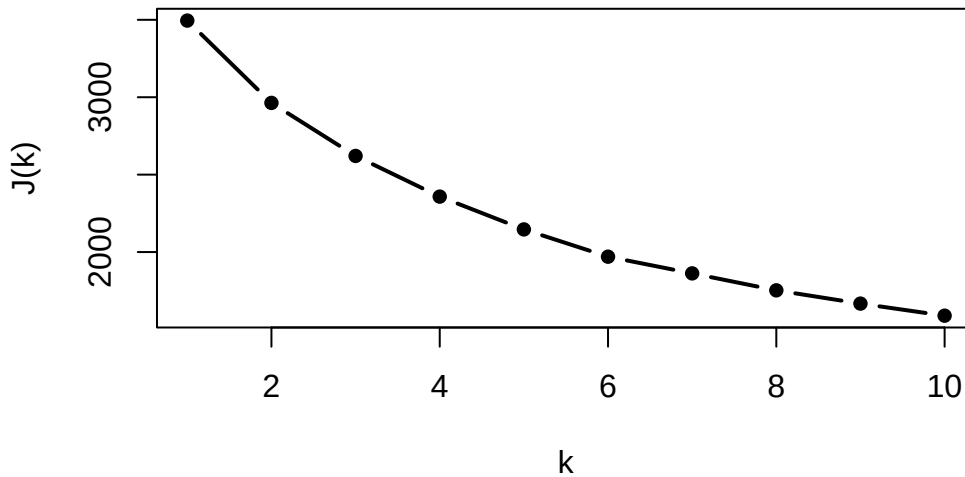
Rather than relying on an arbitrary number of segments, I sought a data-driven approach to discover the true underlying structure of the cohort. I systematically iterated the K-means algorithm, testing every value of k from 1 through 10. Maintaining a consistent random seed prior to each run ensured valid comparisons across all iterations. By plotting the calculated loss J against the number of clusters k , I mapped the trajectory of the variance. Identifying the inflection point in this plot provided a mathematical justification for selecting the optimal number of clusters, balancing model simplicity with group cohesion.

```

set.seed(1120)
J.values = numeric(10)
for(k in 1:10){
  J.values[k] = kmeans(scaled.cov, centers=k, nstart=10)$tot.withinss}

plot(1:10, J.values, type="b", pch=16, lwd=2, xlab="k", ylab="J(k)")

```



The loss curve, as k increases from 1 to 10, does not display a clear elbow or a sharp drop. Instead, the decrease is gradual and progressive, with no evident inflection point where adding clusters stops yielding substantial improvement. Because this decline is almost monotonic, it is difficult to identify an optimal number of clusters from this plot. A reasonable choice might be $k=6$, where the slope appears to flatten slightly, though the plot does not strongly favor any particular value of k . This suggests that the patient cohort does not contain highly compact or naturally well-separated subgroups, but rather exists on a continuous spectrum of clinical profiles.

```

par(mfrow = c(1, 2))
kmeans_res = kmeans(scaled.cov, centers=6, nstart=10)

pca.res = prcomp(scaled.cov, center=FALSE, scale.=FALSE)
pca.scores = pca.res$x[, 1:2]
var.explained = summary(pca.res)$importance[2, 1:2]

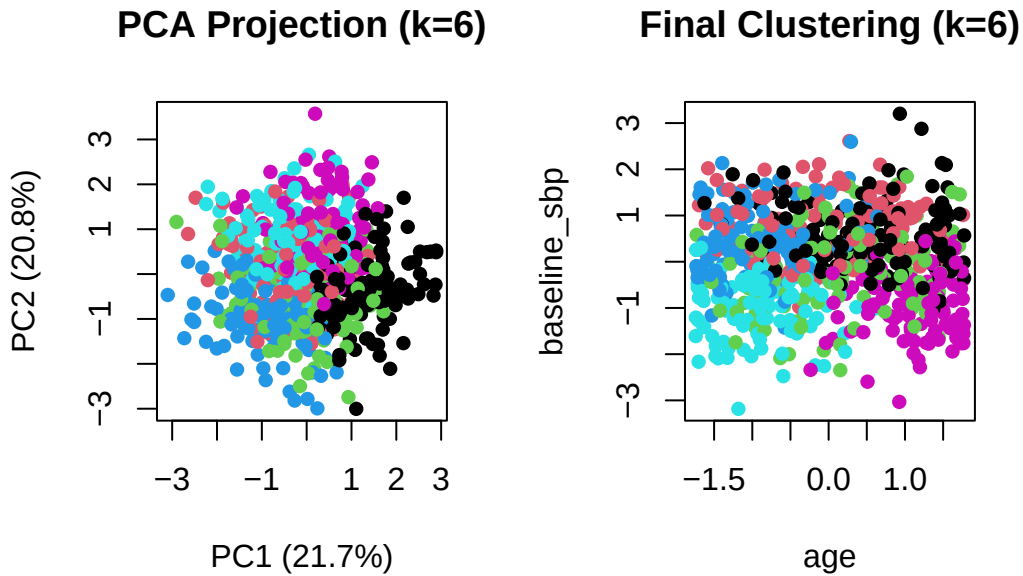
```

```

plot(pca.scores, pch=16, cex=1, col=kmeans_res$cluster,
     xlab=paste0("PC1 (", round(var.explained[1]*100,1), "%)"),
     ylab=paste0("PC2 (", round(var.explained[2]*100,1), "%)"),
     main="PCA Projection (k=6)")

plot(scaled.cov, pch=16, cex=1, col=kmeans_res$cluster,
     main="Final Clustering (k=6)")

```



To evaluate the resulting cluster structure, I generated two side-by-side plots: a Principal Component Analysis (PCA) projection and a standard pairwise scatterplot. The PCA plot mathematically compresses all numerical features to capture the maximum overall variance in a 2D plane, while the standard scatterplot provides a direct look at the relationship between the original scaled covariates. Across both visualizations, the partitioned data reveals heavily interspersed groups rather than distinct, cohesive segments. This persistent visual fragmentation indicates that $k=6$ likely overfits the underlying feature space, effectively dividing noise rather than uncovering genuine clinical patient profiles. Ultimately, the lack of meaningful separation confirms that standard distance-based clustering is not an appropriate exploratory method for this specific dataset.

Predicting Intervention Success (Support Vector Machines)

Shifting the focus from unsupervised pattern discovery to targeted predictive modeling, I sought to classify the clinical success of the hypertension intervention. Rather than predicting the exact reduction in blood pressure, I established a threshold for clinical significance, categorizing a reduction strictly greater than 17 mmHg as a successful outcome (1) and anything lower as unsuccessful (0). With this binary target established, I applied a Support Vector Machine (SVM) leveraging all available clinical, behavioral, and treatment-related covariates.

```
thres = 17
Y_cat = factor(ifelse(data.all$Y < thres,0,1))
data.all.cat = data.all
data.all.cat$Y = Y_cat
train.cat = data.all.cat[select.train,]
test.cat = data.all.cat[-select.train,]
```

To capture potential non-linear relationships between patient profiles and intervention success, we opt for an SVM using a radial kernel. For this initial baseline model, I fixed the cost parameter at 10 and the gamma parameter at 1. By computing the overall classification accuracy across both the training and test datasets, I established a preliminary benchmark of the model's predictive performance before any refinement.

```
library(e1071)
n = dim(data.all.cat)[1]

svm.fit = svm(Y~., data=train.cat, kernel="radial", cost=10, gamma=1,
              scale=TRUE)
suppvector.cost1 = svm.fit$index
prediction.SVCcost1 = predict(svm.fit, newdata=test.cat)

library(caret)
comfMatTest = confusionMatrix(data=prediction.SVCcost1, test.cat$Y)
prediction.SVCcost1.train = predict(svm.fit, newdata=train.cat)
comfMatTrain = confusionMatrix(data=prediction.SVCcost1.train, train.cat$Y)

comfMatTrain
```

Confusion Matrix and Statistics

	Reference	
Prediction	0	1
0	296	0

1 0 194

Accuracy : 1
95% CI : (0.9925, 1)
No Information Rate : 0.6041
P-Value [Acc > NIR] : < 2.2e-16

Kappa : 1

McNemar's Test P-Value : NA

Sensitivity : 1.0000
Specificity : 1.0000
Pos Pred Value : 1.0000
Neg Pred Value : 1.0000
Prevalence : 0.6041
Detection Rate : 0.6041
Detection Prevalence : 0.6041
Balanced Accuracy : 1.0000

'Positive' Class : 0

comfMatTest

Confusion Matrix and Statistics

	Reference	
Prediction	0	1
0	114	54
1	6	36

Accuracy : 0.7143
95% CI : (0.6481, 0.7743)
No Information Rate : 0.5714
P-Value [Acc > NIR] : 1.348e-05

Kappa : 0.375

McNemar's Test P-Value : 1.298e-09

Sensitivity : 0.9500

```

        Specificity : 0.4000
    Pos Pred Value : 0.6786
    Neg Pred Value : 0.8571
        Prevalence : 0.5714
    Detection Rate : 0.5429
    Detection Prevalence : 0.8000
    Balanced Accuracy : 0.6750

'Positive' Class : 0

```

Recognizing that default parameters rarely yield the most generalizable boundaries, I conducted a systematic hyperparameter tuning process. I explored a grid of values, testing cost parameters of 0.1, 1, and 10 alongside gamma values of 1 and 2. Identifying the optimal combination allows to strike a better balance between model complexity and generalization. For this optimized model, I generated a confusion matrix and calculated the final test accuracy to thoroughly evaluate its real-world predictive capabilities. Comparing the two confusion matrices reveals clear overfitting: the model achieves perfect classification accuracy on the training set (100%) but only 71.43% on the test set. With gamma=1, the radial kernel places a very narrow region of influence around each support vector, allowing the decision boundary to curve tightly around individual training points rather than capturing a generalizable pattern.

```

set.seed(1120)
library(caret)
tune.out = tune(svm, Y~., data=train.cat, kernel="radial", probability=TRUE,
               ranges=list(cost=c(0.1,1,10), gamma=c(1,2)))

bestmod.SVC = tune.out$best.model
bestcost = bestmod.SVC$cost
bestgamma = bestmod.SVC$gamma
pred.SVCbest = predict(tune.out$best.model, test.cat)

confusionMatrix(pred.SVCbest, test.cat$Y)

```

Confusion Matrix and Statistics

```

      Reference
Prediction  0   1
    0 114  54
    1   6  36

      Accuracy : 0.7143

```

95% CI : (0.6481, 0.7743)
No Information Rate : 0.5714
P-Value [Acc > NIR] : 1.348e-05

Kappa : 0.375

McNemar's Test P-Value : 1.298e-09

Sensitivity : 0.9500
Specificity : 0.4000
Pos Pred Value : 0.6786
Neg Pred Value : 0.8571
Prevalence : 0.5714
Detection Rate : 0.5429
Detection Prevalence : 0.8000
Balanced Accuracy : 0.6750

'Positive' Class : 0

The tuned SVM achieves an accuracy of 0.7143, identical to the untuned model.

To comprehensively assess whether the tuning process yielded a meaningful improvement, I mapped the Receiver Operating Characteristic (ROC) curves for both the baseline and the optimized models. Comparing these curves provided a robust visual representation of how well each model discriminates between successful and unsuccessful outcomes across all possible probability thresholds, illustrating the true impact of the hyperparameter optimization.

```
library(e1071)
library(ROCR)

svm.fit = svm(Y ~ ., data=train.cat, kernel="radial",
              cost=10, gamma=1, probability=TRUE)
pred = predict(svm.fit, newdata=test.cat, probability=TRUE)
prob = attr(pred, "probabilities")[, "1"]

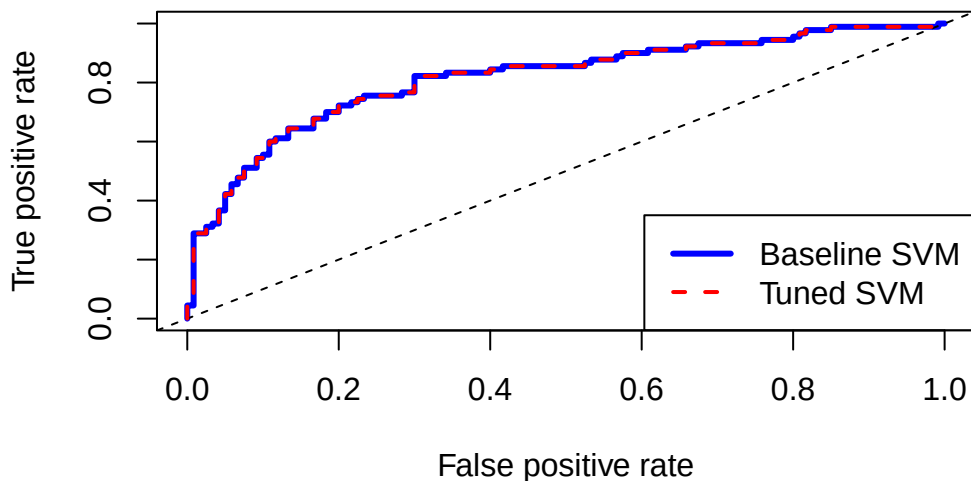
# ROC curve
pred.obj = prediction(prob, test.cat$Y)
perf = performance(pred.obj, "tpr", "fpr")
plot(perf, col="blue", lwd=3)
abline(0, 1, lty=2)

# ROC curve for the tuned model
```

```

prob.best = predict(tune.out$best.model, newdata=test.cat, probability=TRUE)
prob.best.vals = attr(prob.best, "probabilities")[, "1"]
pred.obj.best = prediction(prob.best.vals, test.cat$Y)
perf.best = performance(pred.obj.best, "tpr", "fpr")
plot(perf.best, col="red", lwd=2, lty=2, add=TRUE)
legend("bottomright", legend=c("Baseline SVM", "Tuned SVM"),
      col=c("blue", "red"), lwd=c(3,2), lty=c(1,2))

```



```

# AUC
auc = performance(pred.obj, "auc")
auc <- auc@y.values[[1]]
auc

```

```
[1] 0.8156481
```

The ROC curve, derived from the probability estimates of the baseline SVM, demonstrates solid discriminative ability with an AUC of approximately 0.816. This metric indicates that the classifier effectively ranks successful and unsuccessful interventions, providing strong predictive utility despite its moderate raw test accuracy. Because the subsequent hyperparameter tuning achieved nearly identical performance, the optimized model's ROC curve is virtually indistinguishable from the baseline. Consequently, analyzing the original ROC curve is sufficient.

Non-Linear Effects Modeling (Generalized Additive Models)

Returning to a continuous target variable, I shifted the analytical focus to uncover potential non-linear relationships between the covariates and the reduction in systolic blood pressure. Because standard linear models can miss nuanced physiological or behavioral thresholds, I employed a Generalized Additive Model (GAM). To allow for flexible, non-linear fits without overly complicating the curves, I applied smoothing splines with 3 degrees of freedom across all numerical covariates.

```
library(gam)
library(splines)
gam.model = gam(Y ~ s(age,3) + s(baseline_sbp,3) + s(BMI,3) + s(salt_intake,3)
               + s(adherence,3) + sex + smoker + exercise_level + treatment
               + diabetes, data=train)

# Training set MSE
yhat.gam.train = predict(gam.model, newdata=train)
MSE.gam.train = mean((yhat.gam.train - train$Y)^2)
MSE.gam.train
```

```
[1] 121.845
```

```
# Test set MSE
yhat.gam.test = predict(gam.model, newdata=test)
MSE.gam.test = mean((yhat.gam.test - test$Y)^2)
MSE.gam.test
```

```
[1] 77.18914
```

To establish the baseline performance of this flexible approach, I calculated the Mean Squared Error (MSE) across both the training and test datasets. This evaluation is crucial to ensure that the GAM successfully generalizes to unseen patient profiles rather than simply memorizing the specific fluctuations of the training cohort.

To verify that the observed non-linearities represent true physiological relationships rather than random statistical noise, I evaluated the model's summary table. Because assessing multiple variables simultaneously increases the risk of false discoveries, I applied a Family-Wise Error Rate (FWER) control to adjust the p-values. This rigorous correction allows to confidently isolate which covariates genuinely exhibit significant non-linear effects on patient outcomes.

```
summary(gam.model)
```

```
Call: gam(formula = Y ~ s(age, 3) + s(baseline_sbp, 3) + s(BMI, 3) +  
      s(salt_intake, 3) + s(adherence, 3) + sex + smoker + exercise_level +  
      treatment + diabetes, data = train)
```

Deviance Residuals:

Min	1Q	Median	3Q	Max
-108.81721	-4.71090	0.05131	5.36467	29.45625

(Dispersion Parameter for gaussian family taken to be 127.8471)

Null Deviance: 136155.9 on 489 degrees of freedom

Residual Deviance: 59704.55 on 466.9998 degrees of freedom

AIC: 3791.912

Number of Local Scoring Iterations: NA

Anova for Parametric Effects

	Df	Sum Sq	Mean Sq	F value	Pr(>F)
s(age, 3)	1	15896	15896.3	124.3387	< 2.2e-16 ***
s(baseline_sbp, 3)	1	12013	12012.7	93.9619	< 2.2e-16 ***
s(BMI, 3)	1	4750	4750.2	37.1557	2.286e-09 ***
s(salt_intake, 3)	1	18855	18854.6	147.4777	< 2.2e-16 ***
s(adherence, 3)	1	672	671.9	5.2556	0.02232 *
sex	1	34	33.8	0.2647	0.60717
smoker	1	403	403.0	3.1525	0.07646 .
exercise_level	2	532	265.8	2.0791	0.12620
treatment	2	4681	2340.7	18.3082	2.214e-08 ***
diabetes	1	558	558.2	4.3661	0.03720 *
Residuals	467	59705	127.8		

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Anova for Nonparametric Effects

	Npar	Df	Npar F	Pr(F)
(Intercept)				
s(age, 3)	2	11.287	1.632e-05	***
s(baseline_sbp, 3)	2	0.529	0.5897	
s(BMI, 3)	2	2.284	0.1030	
s(salt_intake, 3)	2	57.288	< 2.2e-16	***
s(adherence, 3)	2	1.134	0.3226	

```
sex
smoker
exercise_level
treatment
diabetes
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
gam.sum = summary(gam.model)
p.param = gam.sum$parametric.anova[, "Pr(>F)"]
p.nonpar = gam.sum$anova$"Pr(F)"
p.nonpar = p.nonpar[!is.na(p.nonpar)]
p.all = c(p.param, p.nonpar)

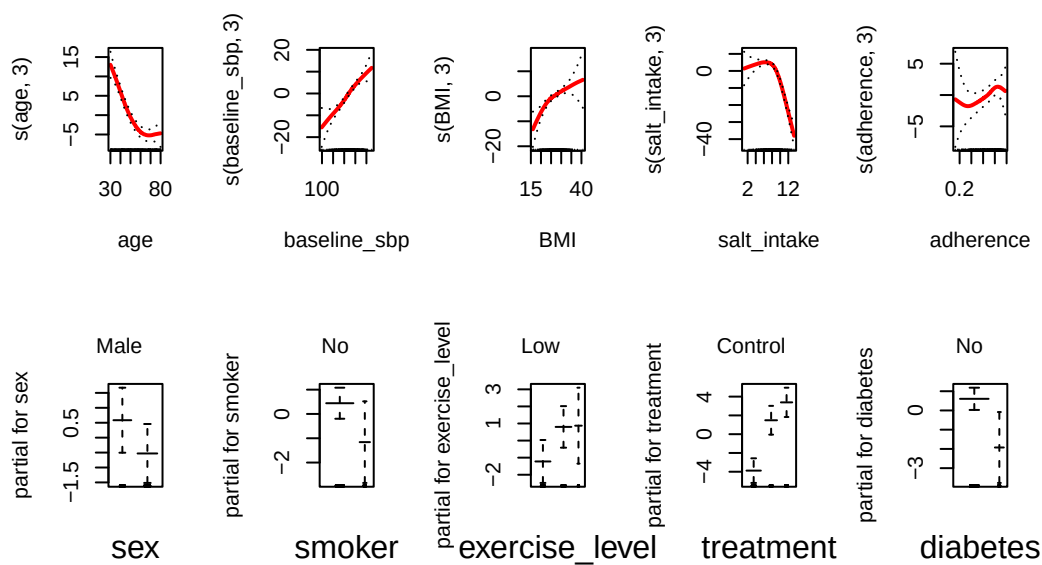
p.bonf = p.adjust(p.all, method="bonferroni")
p.bonf
```

```
[1] 1.380598e-24 3.437174e-19 3.428434e-08 1.657487e-28 3.348045e-01
[6] 1.000000e+00 1.000000e+00 1.000000e+00 3.320819e-07 5.580245e-01
[11]          NA 2.448213e-04 1.000000e+00 1.000000e+00 0.000000e+00
[16] 1.000000e+00
```

After applying Bonferroni correction, only the non-linear components of *age* and *salt_intake* remain statistically significant. The non-linear terms of *baseline_sbp*, *BMI*, and *adherence* lose significance after correction, indicating that their non-linear contribution is weak relative to the stringent FWER threshold. This is consistent with the effect plots showing: clear curvature for *age* and *salt_intake*, near-linear shapes for *baseline_sbp* and *BMI*, minimal variation for adherence. Thus, the GAM suggests the true regression function is partially non-linear, with non-linearity concentrated mainly in age and salt intake.

Guided by these adjusted statistical findings and supported by visual inspections of the partial effect plots, I systematically reduced the model by eliminating the non-significant covariates. By stripping away the noise, I could focus on interpreting the remaining features, extracting a clearer understanding of exactly how these key clinical and behavioral factors influence the final reduction in systolic blood pressure.

```
layout(matrix(1:10, nrow=2, byrow=TRUE))
plot(gam.model, se=TRUE, col="red", lwd=2)
```



```
gam.reduced = update(gam.model, Y ~ s(age,3) + baseline_sbp + BMI
                      + s(salt_intake,3) + adherence + treatment + diabetes,
                      data=train)
```

```
AIC(gam.model, gam.reduced)
```

	df	AIC
gam.model	14	3791.912
gam.reduced	10	3786.453

```
summary(gam.reduced)
```

```
Call: gam(formula = Y ~ s(age, 3) + baseline_sbp + BMI + s(salt_intake,
3) + adherence + treatment + diabetes, data = train)
```

```
Deviance Residuals:
```

Min	1Q	Median	3Q	Max
-107.2509	-4.7632	0.5207	5.6693	31.6731

```
(Dispersion Parameter for gaussian family taken to be 128.9369)
```

Null Deviance: 136155.9 on 489 degrees of freedom
 Residual Deviance: 61502.86 on 476.9998 degrees of freedom
 AIC: 3786.453

Number of Local Scoring Iterations: NA

Anova for Parametric Effects

	Df	Sum Sq	Mean Sq	F value	Pr(>F)
s(age, 3)	1	15947	15947.1	123.6817	< 2.2e-16 ***
baseline_sbp	1	11844	11843.9	91.8579	< 2.2e-16 ***
BMI	1	4714	4713.6	36.5575	2.994e-09 ***
s(salt_intake, 3)	1	19003	19003.0	147.3819	< 2.2e-16 ***
adherence	1	764	764.5	5.9292	0.01526 *
treatment	2	4634	2317.0	17.9699	2.991e-08 ***
diabetes	1	549	549.4	4.2611	0.03954 *
Residuals	477	61503	128.9		

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Anova for Nonparametric Effects

	Npar	Df	Npar F	Pr(F)
(Intercept)				
s(age, 3)	2	11.455	1.383e-05	***
baseline_sbp				
BMI				
s(salt_intake, 3)	2	60.123	< 2.2e-16	***
adherence				
treatment				
diabetes				

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

```
yhat.gam.red = predict(gam.reduced, newdata=test)
MSE.gam.red = mean((yhat.gam.red - test$Y)^2)
```

```
MSE.gam.red
```

```
[1] 82.36546
```

AIC improves (3791.91 -> 3786.45), so the simpler model fits the training data more efficiently. But test MSE gets slightly worse (77.19 -> 82.37), meaning the flattened terms (baseline_sbp, BMI, adherence) still had a little real predictive value, even though it wasn't significant enough

to survive Bonferroni correction. AIC and test MSE don't always agree: here the more complex model generalizes a bit better despite "failing" the significance test.

Interpretation of key effects:

- Age: high and clearly non-linear impact; reductions in systolic BP increase sharply for younger and older patients.
- Salt intake: exhibits a strong non-linear pattern, with worse outcomes at high intake levels.
- Baseline SBP and BMI: linear and significant, showing that patients starting with higher blood pressure experience greater reductions.
- Treatment: DrugA and DrugB outperform control significantly.
- Diabetes: affects response magnitude, with diabetic patients responding less favorably.

Covariate Interactions Analysis (Gradient Boosting)

To further investigate multi-way interactions among patient covariates that simpler models might miss, I implemented a Gradient Boosting machine. For the initial baseline configuration, the model was constructed with 5000 sequential trees ($B=5000$), a conservative learning rate or shrinkage parameter ($\lambda=0.01$), and a fixed interaction depth of 4 ($d=4$).

To establish the predictive accuracy of this specific ensemble architecture, I calculated the Mean Squared Error (MSE) across both the training and test sets. This evaluation provided a fundamental benchmark to measure how well the baseline boosting model generalizes to unseen patient profiles prior to any structural tuning.

```
library(gbm)
B=5000
lambda=0.01
d=4
set.seed(1120)

boost.model = gbm(Y ~ .,
                  data=train,
                  distribution="gaussian",
                  n.trees=B,
                  shrinkage=lambda,
                  interaction.depth=d)

# Train set MSE
yhat.boost.train = predict(boost.model, newdata=train, n.trees=B)
MSE.boost.train = mean((yhat.boost.train - train$Y)^2)
MSE.boost.train
```

```
[1] 6.310776
```

```
# Test set MSE
yhat.boost.test = predict(boost.model, newdata=test, n.trees=B)
MSE.boost.test = mean((yhat.boost.test - test$Y)^2)
MSE.boost.test
```

```
[1] 35.08872
```

Because the depth of the individual trees dictates the maximum level of covariate interaction the model is permitted to capture, I systematically recomputed the model's test MSE across a continuous range of interaction depths, spanning from $d=1$ through $d=7$. Mapping these

error rates let identify the optimal value for d , effectively balancing the model's capacity to learn intricate physiological relationships against the inherent risk of overfitting.

```
d.all = 1:7
MSE.depth = numeric(7)
for(dd in d.all){
  set.seed(1120)
  boost.fit = gbm(Y ~ .,
                  data=train,
                  distribution="gaussian",
                  n.trees=B,
                  shrinkage=lambda,
                  interaction.depth=dd)
  yhat.boost2 = predict(boost.fit, newdata=test, n.trees=B)
  MSE.depth[dd] = mean((yhat.boost2 - test$Y)^2)
}

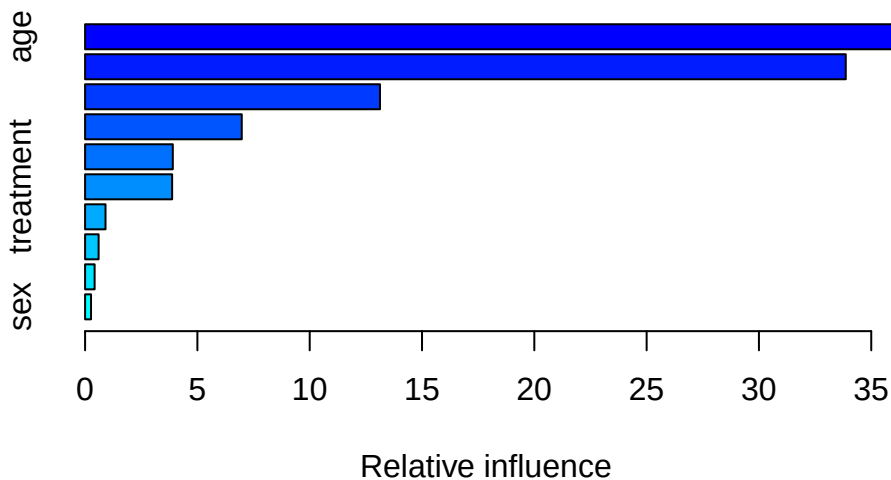
MSE.depth
```

```
[1] 91.12078 34.97618 34.91654 35.08872 35.85230 36.90732 37.00221
```

The smallest test MSE occurs at $\text{depth}=3$ ($\text{MSE}=34.92$). $\text{Depth}=4$, used initially, performs slightly worse. Depth values above 4 increase the test error, indicating overfitting due to overly complex trees. Thus, the best-performing model is obtained with interaction $d=3$, meaning the underlying regression structure likely involves low-order interactions, not purely additive effects, but also not highly complex interaction patterns.

Beyond raw predictive accuracy, I leveraged the boosting framework's internal metrics to extract feature importance scores, directly assessing which covariates prove the most informative overall. Analyzing this relative influence hierarchy provides a clear, data-driven interpretation of which specific behavioral, clinical, or treatment-related factors predominantly drive the reduction in systolic blood pressure within the cohort.

```
summary(boost.model)
```



	var	rel.inf
age	age	36.0529157
salt_intake	salt_intake	33.8648213
baseline_sbp	baseline_sbp	13.1278944
BMI	BMI	6.9743856
adherence	adherence	3.9051428
treatment	treatment	3.8749759
exercise_level	exercise_level	0.9099273
diabetes	diabetes	0.6021578
smoker	smoker	0.4240749
sex	sex	0.2637043

The variable importance plot shows that: *age* and *salt_intake* dominate the model, consistent with GAM results; *baseline_sbp* and *BMI* have moderate influence; *treatment* and *adherence* contribute but to a lesser degree; lifestyle-related categorical predictors (*exercise_level*, *smoker*), *sex*, and *diabetes* have minimal impact. Boosting highlights the same key covariates as GAM, reinforcing that patient age, baseline cardiovascular status, and dietary salt intake drive most of the variability in systolic BP reduction.

Final Evaluation

As a final analytical step, I fitted a standard multiple linear regression model to the entire dataset to establish a highly interpretable benchmark. By computing the test set MSE for this linear approach, I created a fundamental baseline against which the predictive performance of the more complex, non-linear, and ensemble methods could be directly compared.

```
lm.model = lm(Y ~ ., data=train)

yhat.lm = predict(lm.model, newdata=test)
MSE.lm = mean((yhat.lm - test$Y)^2)
MSE.lm
```

```
[1] 111.7485
```

Model comparison:

```
MSE.gam.test
```

```
[1] 77.18914
```

```
MSE.gam.red
```

```
[1] 82.36546
```

```
MSE.boost.test
```

```
[1] 35.08872
```

```
MSE.depth
```

```
[1] 91.12078 34.97618 34.91654 35.08872 35.85230 36.90732 37.00221
```

```
MSE.lm
```

```
[1] 111.7485
```

Conclusion: The Boosting model, optimized with an interaction depth of $d=3$, demonstrates the highest predictive accuracy across all tested methodologies. Boosting successfully captures non-linearities and interactions that the GAM cannot fully represent using low-degree splines.

Based on the relative feature influence extracted from the boosting architecture, the reduction in systolic blood pressure is driven by several distinct clinical and behavioral profiles:

- Primary non-linear drivers: both patient age and daily salt intake exhibit steep, highly non-linear effects on the intervention's success.
- Moderate linear predictors: baseline systolic blood pressure and Body Mass Index (BMI) contribute consistent, linear impacts on the final blood pressure reduction.
- Clinical efficacy: the assigned pharmacological treatment arms demonstrate a meaningful and prominent effect on patient outcomes.
- Marginal covariates: general demographic and broader lifestyle factors contribute only minor predictive value to the overall model.

To further refine predictive performance and ensure the model's robust generalizability to new patient cohorts, subsequent iterations of this research should focus on methodological enhancements. Key improvements would include utilizing rigorous cross-validation to simultaneously tune the shrinkage parameter and the total number of trees, as well as to select the optimal interaction depth with greater statistical precision. Additionally, evaluating model stability through repeated stochastic runs or the integration of bagging techniques would help ensure consistent predictive reliability.